# SQL - MYSQL

Bernardo Caldas

# ELEMENTOS DOS BANCOS DE DADOS

## 1 ARQUIVOS DE BANCOS DE DADOS



Conjunto de dados armazenados em estruturas físicas (**arquivos**).

## 2 SGBD



Sistema Gerenciador de Bancos de Dados. Software que permite criar, manipular e controlar os dados de forma eficiente e segura.

## 3 LINGUAGEM DE CONVERSAÇÃO




Linguagem usada para se comunicar com bancos de dados relacionais.

## 3.1 SQL (1974 - IBM)

**ISO e ANSI**
(linguagem padronizada)

**Dialetos do SQL**

- transact-sql (Microsoft)
- PL/sql (oracle)

**Grupos de comandos**

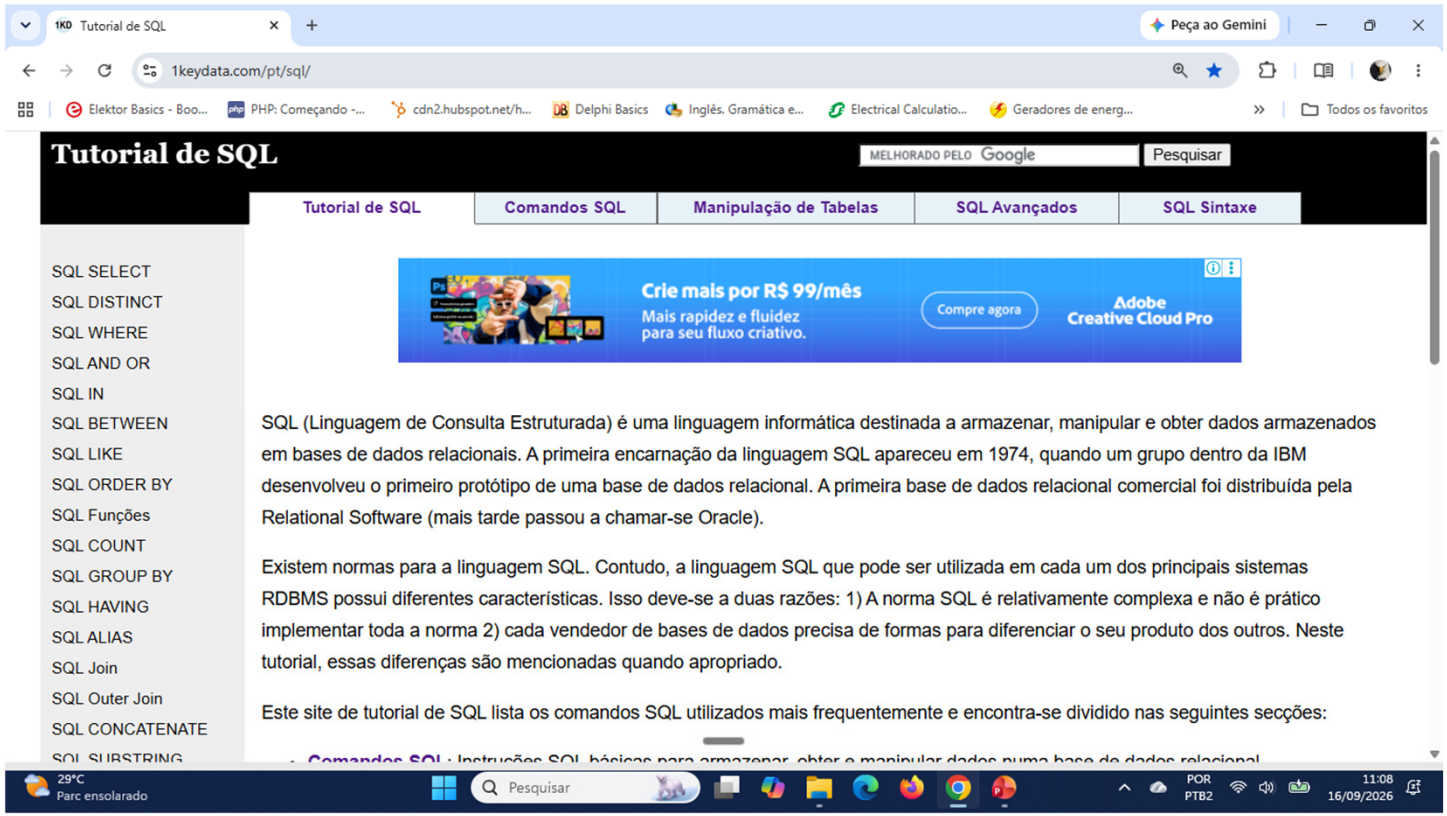
| | | | |
|---|---|---|---|
| **DDL** (data definition language create, alter, drop) | Define a estrutura dos objetos do banco de dados. | CREATE, ALTER, DROP | => DBA |
| **DCL** (data control language trant, revoke, deny) | Controla o acesso aos dados. | GRANT, REVOKE, DENY | => DBA |
| **DML** (data manipulation language select insert, update,delete) | Manipula os dados armazenados no banco de dados. | SELECT, INSERT, UPDATE, DELETE | => USER |

# SUPERMERCADO

https://www.youtube.com/watch?v=Z6aG4Ca0sxY

https://www.youtube.com/watch?v=pjgT7laPgcE

# TUTORIAL


Tutorial de SQL
Peça ao Gemini
1keydata.com/pt/sql/
Elektor Basics - Boo...
PHP: Começando -...
cdn2.hubspot.net/h...
Delphi Basics
Inglês. Gramática e...
Electrical Calculatio...
Geradores de energ...
Todos os favoritos
Tutorial de SQL
MELHORADO PELO Google
Pesquisar
Tutorial de SQL
Comandos SQL
Manipulação de Tabelas
SQL Avançados
SQL Sintaxe
SQL SELECT
SQL DISTINCT
SQL WHERE
SQL AND OR
SQL IN
SQL BETWEEN
SQL LIKE
SQL ORDER BY
SQL Funções
SQL COUNT
SQL GROUP BY
SQL HAVING
SQL ALIAS
SQL Join
SQL Outer Join
SQL CONCATENATE
SQL SUBSTRING
Crie mais por R$ 99/mês
Mais rapidez e fluidez
para seu fluxo criativo.
Compre agora
Adobe
Creative Cloud Pro
SQL (Linguagem de Consulta Estruturada) é uma linguagem informática destinada a armazenar, manipular e obter dados armazenados em bases de dados relacionais. A primeira encarnação da linguagem SQL apareceu em 1974, quando um grupo dentro da IBM desenvolveu o primeiro protótipo de uma base de dados relacional. A primeira base de dados relacional comercial foi distribuída pela Relational Software (mais tarde passou a chamar-se Oracle).
Existem normas para a linguagem SQL. Contudo, a linguagem SQL que pode ser utilizada em cada um dos principais sistemas RDBMS possui diferentes características. Isso deve-se a duas razões: 1) A norma SQL é relativamente complexa e não é prático implementar toda a norma 2) cada vendedor de bases de dados precisa de formas para diferenciar o seu produto dos outros. Neste tutorial, essas diferenças são mencionadas quando apropriado.
Este site de tutorial de SQL lista os comandos SQL utilizados mais frequentemente e encontra-se dividido nas seguintes secções:
29°C
Parc ensolarado
Pesquisar
POR
PTB2
11:08
16/09/2026

# SQL

## Resumo

| Linguagem | Objetivo | Principais comandos |
| --- | --- | --- |
| DDL | Define a estrutura do banco | `CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME` |
| DML | Manipula os dados | `INSERT`, `UPDATE`, `DELETE` |
| DQL | Consulta os dados | `SELECT` |
| DCL | Controla permissões | `GRANT`, `REVOKE` |
| TCL | Controla transações | `COMMIT`, `ROLLBACK`, `SAVEPOINT` |

# **DDL - Data Definition Language**
(Linguagem de Definição de Dados)

# DDL

| Comando | Para que serve | Exemplo |
|---|---|---|
| **CREATE** | Cria uma tabela, banco de dados, índice ou visão. | `CREATE TABLE Cliente (...);` |
| **ALTER** | Altera a estrutura de uma tabela existente. | `ALTER TABLE Cliente ADD telefone VARCHAR(20);` |
| **DROP** | Exclui definitivamente uma tabela ou outro objeto. | `DROP TABLE Cliente;` |
| **TRUNCATE** | Remove todos os registros da tabela, mantendo sua estrutura. | `TRUNCATE TABLE Cliente;` |
| **RENAME** *(ou ALTER TABLE ... RENAME)* | Renomeia uma tabela. | `RENAME TABLE Cliente TO Clientes;` |
| **COMMENT** *(Oracle/PostgreSQL)* | Adiciona comentários em tabelas e colunas. | `COMMENT ON TABLE Cliente IS 'Cadastro de clientes';` |

# DDL

| Comando | Atua na estrutura? | Remove dados? | Mantém a tabela? |
|---|---|---|---|
| CREATE | ✔ Sim | Não | Cria a tabela |
| ALTER | ✔ Sim | Não | Sim |
| DROP | ✔ Sim | ✔ Sim | ❌ Não |
| TRUNCATE | Não altera a estrutura | ✔ Sim | ✔ Sim |
| RENAME | ✔ Sim | Não | ✔ Sim |

## CREATE DATABASE nome_do_banco;

- **CREATE DATABASE vendas;**
  - Após criar o banco, é necessário selecioná-lo para começar a criar tabelas: **USE vendas;**
  - **CREATE DATABASE IF NOT EXISTS vendas;**

CREATE TABLE nome_tabela ( coluna1 tipo [restrições], ...
[PRIMARY KEY (...)]);

- **CREATE TABLE cliente (**
  **id_cliente INT PRIMARY KEY,**
  **nome VARCHAR(100),**
  **cidade VARCHAR(50),**
  **idade INT );**

CREATE TABLE nome_tabela ( coluna1 tipo [restrições], ...
[PRIMARY KEY (...)]);

- **CREATE TABLE IF NOT EXISTS Store_Information (**
  **Store_Name VARCHAR(50) NOT NULL,**
  **Sales INT NOT NULL,**
  **Txn_Date DATE NOT NULL );**

CREATE TABLE nome_tabela ( coluna1 tipo [restrições], ...
[PRIMARY KEY (...)]);

- **CREATE TABLE funcionario (**
  **id_funcionario INT AUTO_INCREMENT,**
  **nome VARCHAR(100) NOT NULL,**
  **cpf CHAR(11) UNIQUE,**
  **salario DECIMAL(10,2),**
  **data_admissao DATE,**
  **ativo BOOLEAN DEFAULT TRUE,**
  **PRIMARY KEY (id_funcionario) );**

CREATE TABLE nome_tabela ( coluna1 tipo [restrições], ...
[PRIMARY KEY (...)]);

### Estrutura do comando

| Elemento | Função |
| --- | --- |
| `CREATE TABLE` | Cria uma nova tabela. |
| `nome_tabela` | Nome da tabela. |
| `coluna` | Nome do campo. |
| `tipo` | Tipo de dado ( `INT` , `VARCHAR` , `DATE` , etc.). |
| `PRIMARY KEY` | Define a chave primária. |
| `NOT NULL` | Impede valores nulos. |
| `UNIQUE` | Não permite valores repetidos. |
| `DEFAULT` | Define um valor padrão. |
| `AUTO_INCREMENT` | Incrementa automaticamente valores inteiros. |

CREATE TABLE nome_tabela ( coluna1 tipo [restrições], ... [PRIMARY KEY (...)]);

**Tipos de dados mais usados**

| Tipo | Exemplo |
|---|---|
| `INT` | 10 |
| `VARCHAR(100)` | "Maria" |
| `CHAR(2)` | "PE" |
| `DECIMAL(10,2)` | 1234.56 |
| `DATE` | 2026-08-01 |
| `DATETIME` | 2026-08-01 14:30:00 |
| `BOOLEAN` | TRUE ou FALSE |

## ALTER TABLE nome_tabela
## ação;

| Operação | Sintaxe |
| --- | --- |
| Adicionar coluna | `ALTER TABLE ... ADD` |
| Alterar coluna | `ALTER TABLE ... MODIFY` |
| Renomear coluna | `ALTER TABLE ... RENAME COLUMN` |
| Excluir coluna | `ALTER TABLE ... DROP COLUMN` |
| Adicionar chave primária | `ALTER TABLE ... ADD PRIMARY KEY` |
| Adicionar chave estrangeira | `ALTER TABLE ... ADD CONSTRAINT ... FOREIGN KEY` |
| Renomear tabela | `ALTER TABLE ... RENAME TO` |

ALTER TABLE nome_tabela
ação;

- Os mais utilizados:

1. **ADD** (adicionar coluna);
2. **MODIFY** (alterar tipo ou tamanho de uma coluna);
3. **RENAME COLUMN** (renomear coluna);
4. **DROP COLUMN** (remover coluna).

# ALTER TABLE nome_tabela ação;

### 1. Adicionar uma coluna

SQL

```sql
ALTER TABLE nome_tabela
ADD nome_coluna tipo;
```

Exemplo:

SQL

```sql
ALTER TABLE Cliente
ADD telefone VARCHAR(20);
```

### 2. Alterar o tipo de uma coluna

SQL

```sql
ALTER TABLE nome_tabela
MODIFY nome_coluna novo_tipo;
```

Exemplo:

SQL

```sql
ALTER TABLE Cliente
MODIFY telefone VARCHAR(30);
```

### 3. Renomear uma coluna (MySQL 8.0+)

SQL

```sql
ALTER TABLE nome_tabela
RENAME COLUMN nome_antigo TO nome_novo;
```

Exemplo:

SQL

```sql
ALTER TABLE Cliente
RENAME COLUMN telefone TO celular;
```

# ALTER TABLE nome_tabela ação;

### 4. Adicionar uma coluna depois de determinada coluna

**Sintaxe:**

SQL

```sql
ALTER TABLE nome_tabela
ADD nome_coluna tipo AFTER coluna_existente;
```

**Exemplo:**

SQL

```sql
ALTER TABLE Cliente
ADD celular VARCHAR(20) AFTER telefone;
```

### 5. Adicionar uma coluna antes de determinada coluna

O MySQL **não possui** `BEFORE`. Para colocar antes de uma coluna, usamos `AFTER` indicando a **coluna anterior.**

Se temos:

`id | nome | telefone`

e queremos `email` **antes de telefone:**

SQL

```sql
ALTER TABLE Cliente
ADD email VARCHAR(100) AFTER nome;
```

# ALTER TABLE nome_tabela
# ação;

### 4. Excluir uma coluna

SQL

```sql
ALTER TABLE nome_tabela
DROP COLUMN nome_coluna;
```

Exemplo:

SQL

```sql
ALTER TABLE Cliente
DROP COLUMN telefone;
```

### 5. Adicionar uma chave primária

SQL

```sql
ALTER TABLE nome_tabela
ADD PRIMARY KEY (nome_coluna);
```

Exemplo:

SQL

```sql
ALTER TABLE Cliente
ADD PRIMARY KEY (id);
```

### 7. Renomear a tabela

SQL

```sql
ALTER TABLE nome_tabela
RENAME TO novo_nome;
```

Exemplo:

SQL

```sql
ALTER TABLE Cliente
RENAME TO Clientes;
```

# ALTER TABLE nome_tabela ação;

### 6. Adicionar uma chave estrangeira

SQL

```sql
ALTER TABLE nome_tabela
ADD CONSTRAINT fk_nome
FOREIGN KEY (coluna)
REFERENCES outra_tabela(coluna);
```

### 1. Criar a tabela Cliente

SQL

```sql
CREATE TABLE Cliente (
    cod_cliente INT PRIMARY KEY,
    nome VARCHAR(100)
);
```

### 2. Criar a tabela Pedido (sem a chave estrangeira)

SQL

```sql
CREATE TABLE Pedido (
    cod_pedido INT PRIMARY KEY,
    data_pedido DATE,
    cod_cliente INT
);
```

### 3. Adicionar a chave estrangeira com `ALTER TABLE`

SQL

```sql
ALTER TABLE Pedido
ADD CONSTRAINT fk_pedido_cliente
FOREIGN KEY (cod_cliente)
REFERENCES Cliente(cod_cliente);
```

# DELETAR TABELAS – ATENÇÃO – MUITO CUIDADO

| Comando | Categoria | O que faz |
|---|---|---|
| `DELETE` | DML | Exclui linhas; pode usar `WHERE` |
| `TRUNCATE` | DDL* | Remove todas as linhas rapidamente; não usa `WHERE` |
| `DROP` | DDL | Remove a própria tabela |

</> SQL

```sql
TRUNCATE TABLE Cliente;
```

</> SQL

```sql
DROP DATABASE escola;
```

# **DQL - Data Query Language**
(Linguagem de Consulta de Dados)

# DQL

| Comando | O que faz | Exemplo |
|---|---|---|
| SELECT | Consulta dados de uma tabela. | `SELECT * FROM Produto;` |
| DISTINCT | Elimina valores repetidos. | `SELECT DISTINCT cidade FROM Cliente;` |
| WHERE | Filtra registros conforme uma condição. | `WHERE salario > 3000` |
| AND | Exige que todas as condições sejam verdadeiras. | `idade > 18 AND sexo='F'` |
| OR | Basta uma condição ser verdadeira. | `cidade='Recife' OR cidade='Olinda'` |
| IN | Procura valores em uma lista. | `WHERE estado IN ('PE','PB')` |

# DQL

| | | |
|---|---|---|
| **BETWEEN** | Filtra um intervalo de valores. | `WHERE preco BETWEEN 10 AND 50` |
| **LIKE** | Pesquisa utilizando padrões (% e _). | `LIKE 'A%'` |
| **ORDER BY** | Ordena o resultado da consulta. | `ORDER BY nome ASC` |
| **ASC** | Ordenação crescente (padrão). | `ORDER BY nome ASC` |
| **DESC** | Ordenação decrescente. | `ORDER BY salario DESC` |
| **AVG()** | Calcula a média. | `AVG(salario)` |
| **COUNT()** | Conta registros. | `COUNT(*)` |
| **SUM()** | Soma valores. | `SUM(valor)` |

# DQL

| | | |
|---|---|---|
| **MIN()** | Retorna o menor valor. | `MIN(preco)` |
| **MAX()** | Retorna o maior valor. | `MAX(preco)` |
| **GROUP BY** | Agrupa registros para cálculos. | `GROUP BY cidade` |
| **HAVING** | Filtra grupos após o agrupamento. | `HAVING COUNT(*) > 5` |
| **AS (Alias)** | Cria um nome temporário para coluna ou tabela. | `SUM(venda) AS Total` |
| **INNER JOIN** | Retorna apenas registros existentes nas duas tabelas. | Clientes com pedidos. |
| **LEFT JOIN** | Retorna todos da tabela da esquerda e os correspondentes da direita. | Todos os clientes, tenham ou não pedidos. |

# DQL

| | | |
|---|---|---|
| **RIGHT JOIN** | Retorna todos da tabela da direita e os correspondentes da esquerda. | Todos os departamentos, tenham ou não funcionários. |

# SELECT "nome_coluna" FROM "nome_tabela";

- **SELECT * FROM Store_Information;**

**Resultado esperado**

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 1999-01-05 |
| San Diego | 250 | 1999-01-07 |
| Los Angeles | 300 | 1999-01-08 |
| Boston | 700 | 1999-01-08 |

SELECT DISTINCT "nome_coluna"
FROM "nome_tabela";

- **SELECT DISTINCT Store_Name FROM Store_Information;**

Tabela ***Store_Information***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |
| Los Angeles | 300 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

***Resultado:***

```
Store_Name
Los Angeles
San Diego
Boston
```

# SELECT "nome_coluna"
# FROM "nome_tabela “ WHERE "condição";

- **SELECT Store_Name FROM Store_Information WHERE Sales > 1000;**

Tabela ***Store_Information***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |
| Los Angeles | 300 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

***Resultado:***

| Store_Name |
|---|
| Los Angeles |

SELECT "nome_coluna“ FROM "nome_tabela"
WHERE "condição simples"
{[AND|OR] "condição simples"}+;

- **SELECT Store_Name FROM Store_Information WHERE Sales > 1000 OR (Sales < 500 AND Sales > 275);**

Tabela ***Store_Information***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |
| Los Angeles | 300 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

***Resultado:***

| Store_Name |
|---|
| Los Angeles |
| San Francisco |

SELECT "nome_coluna“ FROM "nome_tabela"
WHERE "nome_coluna" IN ('valor1', 'valor2', ...);

- **SELECT * FROM Store_Information WHERE Store_Name IN ('Los Angeles', 'San Diego');**

Tabela ***Store_Information***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |
| Los Angeles | 300 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

***Resultado:***

```
Store_Name  Sales Txn_Date
Los Angeles  1500 05-Jan-1999
San Diego     250 07-Jan-1999
```

SELECT "nome_coluna“ FROM "nome_tabela"
WHERE "nome_coluna" BETWEEN ''valor1' AND ''valor2';

- **SELECT * FROM Store_Information WHERE Txn_Date BETWEEN ‘1999-01-06' AND ‘1999-01-10';**

Tabel ***Store_Information***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |
| San Francisco | 300 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

***Resultado:***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| San Diego | 250 | 07-Jan-1999 |
| San Francisco | 300 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

SELECT "nome_coluna“ FROM "nome_tabela"
WHERE "nome_coluna" LIKE {PATTERN};

- **SELECT * FROM Store_Information WHERE Store_Name LIKE '%AN%';**

{PATTERN} normalmente é composto por caracteres universais. A seguir são apresentados alguns exemplos:

- 'A_Z': Todas as cadeias que começam por 'A', outro caráter, e terminam em 'Z'. Por exemplo, 'ABZ' e 'A2Z' iriam ambas satisfazer a condição, enquanto 'AKKZ' não iria (porque existem dois caracteres entre A e Z em vez de um).
- 'ABC%': Todas as cadeias que começam por 'ABC'. Por exemplo, 'ABCD' e 'ABCABC' iriam satisfazer a condição.
- '%XYZ': Todas as cadeias que terminam em 'XYZ'. Por exemplo, 'WXYZ' e 'ZZXYZ' iriam satisfazer a condição.
- '%AN%': Todas as cadeias que contêm o padrão 'AN' em qualquer local. Por exemplo, 'LOS ANGELES' e 'SAN FRANCISCO' iriam satisfazer a condição.

SELECT "nome_coluna“ FROM "nome_tabela"
WHERE "nome_coluna" LIKE {PATTERN};

- **SELECT * FROM Store_Information WHERE Store_Name LIKE '%AN%';**

Tabel ***Store_Information***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |
| San Francisco | 300 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

***Resultado:***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| LOS ANGELES | 1500 | 05-Jan-1999 |
| SAN DIEGO | 250 | 07-Jan-1999 |
| SAN FRANCISCO | 300 | 08-Jan-1999 |

```
SELECT "nome_coluna"
FROM "nome_tabela"[WHERE "condição"]
ORDER BY "nome_coluna" [ASC, DESC];
```

- Os símbolos [ ] significam que a instrução **WHERE** é opcional. Contudo, se existir uma cláusula **WHERE**, esta precede a cláusula **ORDER BY**. **ASC** significa que os resultados serão apresentados por ordem ascendente e **DESC** significa que os resultados serão apresentados por ordem descendente. Se não for especificada qualquer uma, o padrão é **ASC**.

- É possível ordenar por mais do que uma coluna. Nesse caso, a cláusula **ORDER BY** acima torna-se
  - **ORDER BY "nome_coluna1" [ASC, DESC], "nome_coluna2" [ASC, DESC]**

SELECT "nome_coluna"
FROM "nome_tabela"[WHERE "condição"]
ORDER BY "nome_coluna" [ASC, DESC];

- **SELECT Store_Name, Sales, Txn_Date FROM Store_Information ORDER BY Sales DESC;**

Tabela ***Store_Information***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |
| San Francisco | 300 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

***Resultado:***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |
| San Francisco | 300 | 08-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |

**SELECT "tipo de função" ("nome_coluna") FROM "nome_tabela";**

- A linguagem SQL tem várias funções aritméticas, sendo: **AVG, COUNT, MAX, MIN, SUM**

# SELECT "tipo de função" ("nome_coluna") FROM "nome_tabela";

- A linguagem SQL tem várias funções aritméticas, sendo: **AVG, COUNT, MAX, MIN, SUM**

- **SELECT SUM (Sales) FROM Store_Information;**

Tabela ***Store_Information***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |
| Los Angeles | 300 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

***Resultado:***

```
SUM (Sales)
    2750
```

# SELECT COUNT("nome_coluna") FROM "nome_tabela";

- **SELECT COUNT (Store_Name) FROM Store_Information;**

Tabela ***Store_Information***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |
| San Francisco | 300 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

***Resultado:***

```
COUNT (Store_Name)
4
```

- **SELECT COUNT (DISTINCT Store_Name) FROM Store_Information;**

***Resultado:***

```
COUNT (DISTINCT Store_Name)
3
```

**SELECT "nome_coluna1", SUM("nome_coluna2")**
**FROM "nome_tabela“ GROUP BY "nome_coluna1"**
**HAVING (condição da função aritmética);**

- **SELECT Store_Name, SUM(Sales) FROM Store_Information GROUP BY Store_Name HAVING SUM (Sales) > 1500;**

Tabela ***Store_Information***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |
| San Francisco | 300 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

***Resultado:***

| Store_Name | SUM(Sales) |
|---|---|
| Los Angeles | 1800 |

A diferença entre `WHERE` e `HAVING` é:

- **WHERE** → filtra **linhas** antes do agrupamento.
- **HAVING** → filtra **grupos** depois do agrupamento.

```
SELECT "alias_tabela"."nome_coluna1" "alias_coluna"
FROM "nome_tabela" "alias_tabela";
```

- **SELECT A1.Store_Name Store, SUM(A1.Sales) 'Total Sales' FROM Store_Information A1 GROUP BY A1.Store_Name;**

Tabela ***Store_Information***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |
| Los Angeles | 300 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

***Resultado:***

| Store | Total Sales |
|---|---|
| Los Angeles | 1800 |
| San Diego | 250 |
| Boston | 700 |

No MySQL

O correto e mais recomendado é:

```sql
SELECT A1.Store_Name AS Store,
       SUM(A1.Sales) AS `Total Sales`
FROM Store_Information AS A1
GROUP BY A1.Store_Name;
```

# SQL > Comandos SQL > Join

- **INNER JOIN – Retorna apenas os registros que possuem correspondência entre as duas tabelas.**
  - **SELECT * FROM Store_Information S INNER JOIN Geography G ON S.Store_Name = G.Store_Name;**

Tabela ***Store_Information***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |
| Los Angeles | 300 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

Tabela ***Geography***

| Region_Name | Store_Name |
|---|---|
| East | Boston |
| East | New York |
| West | Los Angeles |
| West | San Diego |

| Store_Name | Sales | Txn_Date | Region_Name | Store_Name |
|---|---|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 | West | Los Angeles |
| San Diego | 250 | 07-Jan-1999 | West | San Diego |
| Los Angeles | 300 | 08-Jan-1999 | West | Los Angeles |
| Boston | 700 | 08-Jan-1999 | East | Boston |

# SQL > Comandos SQL > Join

- **LEFT JOIN (ou LEFT OUTER JOIN) –** Retorna todos os registros da tabela da esquerda e os correspondentes da tabela da direita.
  - **SELECT S.Store_Name, G.Region_Name, S.Sales, S.Txn_Date FROM Store_Information S LEFT JOIN Geography G ON S.Store_Name = G.Store_Name;**

Tabela ***Store_Information***

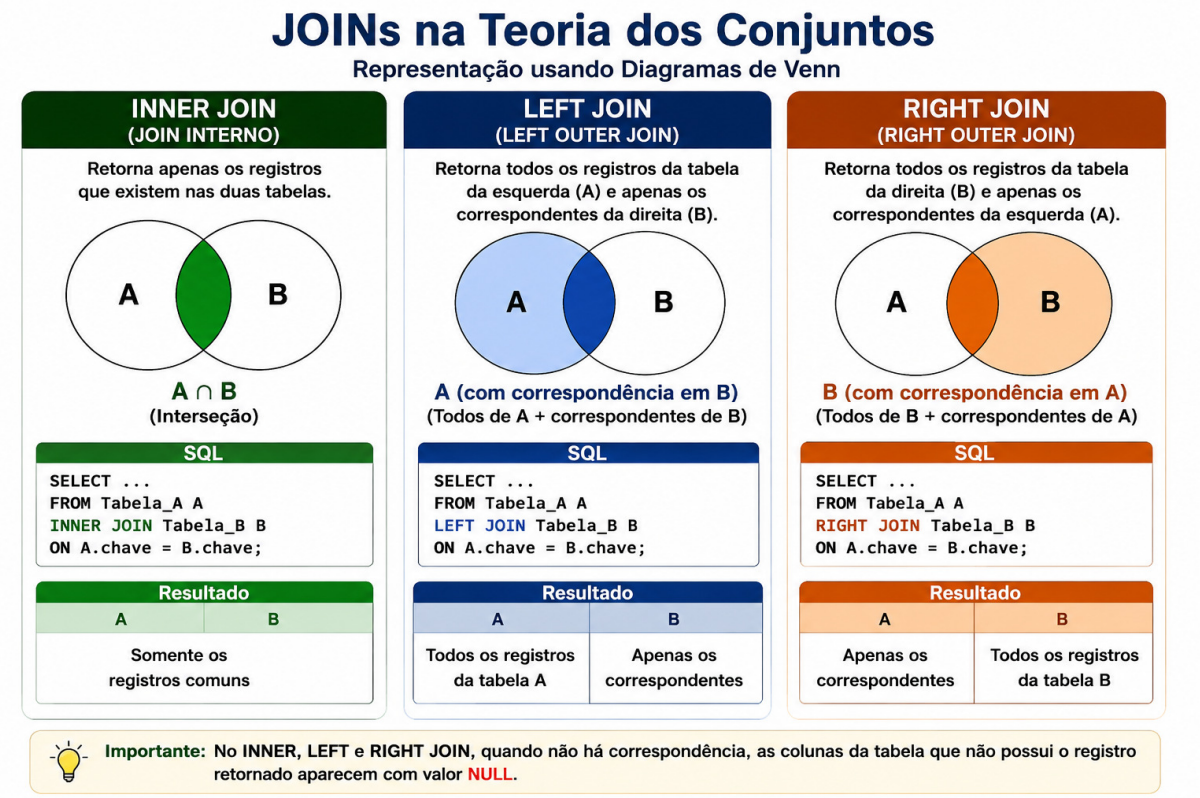
| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |
| Los Angeles | 300 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

Tabela ***Geography***

| Region_Name | Store_Name |
|---|---|
| East | Boston |
| East | New York |
| West | Los Angeles |
| West | San Diego |

| Store_Name | Region_Name | Sales | Txn_Date |
|---|---|---|---|
| Los Angeles | West | 1500 | 1999-01-05 |
| San Diego | West | 250 | 1999-01-07 |
| Los Angeles | West | 300 | 1999-01-08 |
| Boston | East | 700 | 1999-01-08 |

# SQL > Comandos SQL > Join



**DML - Data Manipulation Language**
(Linguagem de Manipulação de Dados).

# DML - Data Manipulation Language

| Comando | Função |
| --- | --- |
| `INSERT` | Insere registros |
| `UPDATE` | Altera registros existentes |
| `DELETE` | Exclui registros |

# SQL > Manipulação de Tabelas > Update

Tabela ***Store_Information***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |
| Los Angeles | 300 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

```
UPDATE Store_Information
SET Sales = 500
WHERE Store_Name = 'Los Angeles'
AND Txn_Date = '08-Jan-1999';
```

```
UPDATE "nome_tabela"
SET "coluna 1" = [novo valor]
WHERE "condição";
```

Tabela ***Store_Information***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |
| Los Angeles | 500 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

# SQL > Manipulação de Tabelas > Insert Into

```
INSERT INTO "nome_tabela" ("coluna 1", "coluna 2", ...)
VALUES ("valor 1", "valor 2", ...);
```

Tabela ***Store_Information***

| Nome da Coluna | Tipo de Dados |
|---|---|
| Store_Name | char(50) |
| Sales | float |
| Txn_Date | datetime |

```
INSERT INTO Store_Information (Store_Name, Sales, Txn_Date)
VALUES ('Los Angeles', 900, '10-Jan-1999' );
```

# SQL > Manipulação de Tabelas > Delete From

```
DELETE FROM "nome_tabela"
WHERE "condição";
```

```
DELETE FROM Store_Information
WHERE Store_Name = 'Los Angeles';
```

Tabela ***Store_Information***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| Los Angeles | 1500 | 05-Jan-1999 |
| San Diego | 250 | 07-Jan-1999 |
| Los Angeles | 300 | 08-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

Tabela ***Store_Information***

| Store_Name | Sales | Txn_Date |
|---|---|---|
| San Diego | 250 | 07-Jan-1999 |
| Boston | 700 | 08-Jan-1999 |

**DCL - Data Control Language**
(Linguagem de Controle de Dados).

# SQL > GRANT & REVOKE

| Comando | Função |
| --- | --- |
| `GRANT` | Concede privilégios a um usuário |
| `REVOKE` | Retira privilégios de um usuário |

# SQL > GRANT & REVOKE

```sql
GRANT SELECT, INSERT
ON bd_2026_2.*
TO 'joao'@'localhost';
```

O que esse comando faz:

- `GRANT` → concede permissões.
- `SELECT, INSERT` → permite **consultar** e **inserir** dados.
- `ON bd_2026_2.*` → essas permissões valem para **todas as tabelas** do banco `bd_2026_2`.
- `TO 'joao'@'localhost'` → concede as permissões ao usuário `joao`, acessando pelo próprio computador.

# SQL > GRANT & REVOKE

```sql
REVOKE INSERT, UPDATE
ON bd_2026_2.*
FROM 'usuario'@'localhost';
```

- `REVOKE` → retira permissões.
- `INSERT, UPDATE` → remove o direito de inserir e alterar dados.
- `ON bd_2026_2.*` → em todas as tabelas do banco `bd_2026_2`.
- `FROM 'usuario'@'localhost'` → do usuário `usuario`.

# SQL > GRANT & REVOKE

- **Criar um novo usuário chamado ‘usuário’ e define a senha**
- **‘localhost’ indica que esse usuário se conecta localmente ao servidor**
- **‘@’ separa o usuário do local onde ele pode se conectar**

```SQL
CREATE USER 'usuario'@'localhost'
IDENTIFIED BY 'senha';
```

**sem espaço entre** ‘usuario', @ e 'localhost'

# SQL > GRANT & REVOKE

```sql
ALTER USER 'usuario'@'localhost'
IDENTIFIED BY 'nova_senha';
```

Altera as informações do usuário.

Nesse exemplo, troca a senha atual pela senha `nova_senha`.

# SQL > GRANT & REVOKE

```sql
DROP USER 'usuario'@'localhost';
```

Exclui o usuário do MySQL.

Depois desse comando, o usuário `usuario` deixa de existir no servidor.

# SQL > GRANT & REVOKE

```sql
SHOW GRANTS FOR 'usuario'@'localhost';
```

Mostra quais privilégios foram concedidos ao usuário.

```
SELECT
INSERT
UPDATE
DELETE
```

## Dúvidas?

# Obrigado